

Comprehensive Java Guide

Batch 1 & 2: Foundations, Memory Architecture, Data Types & OOP Principles

Tailored for UNIBEN Exams

Part 1: The Java Environment and Philosophy

1. Platform Independence

Java is unique because it does not compile directly into machine code for a specific CPU. Instead, it follows a two-step process that enables the famous Write Once, Run Anywhere guarantee.

- **Compilation:** The javac compiler translates your .java source file into an intermediate form called Bytecode. This Bytecode is stored in a .class file and is not tied to any specific machine.
- **Interpretation:** The Bytecode is platform-neutral. It can be moved to any machine and run there.
- **The JVM:** The Java Virtual Machine is the platform-dependent component. It reads the universal Bytecode and translates it into native instructions that the local operating system (Windows, macOS, Linux) can execute.

2. Java Buzzwords

- **Robustness:** Java is engineered to be resilient against crashes. It achieves this by removing explicit pointers (which prevents memory corruption), enforcing Strong Type Checking at compile time, and providing Automatic Garbage Collection to manage memory cleanup without programmer intervention.
- **Architectural Neutrality:** Unlike C or C++, where an int might occupy 2 bytes on one machine and 4 bytes on another, Java fixes the size of every primitive type regardless of the underlying hardware architecture. This eliminates platform-specific data bugs.

Part 2: Java Memory Architecture

Understanding where data lives in memory is essential for answering questions about variable lifetimes, scope, and garbage collection in exams.

1. The Stack

- **What lives here:** Local variables (also called automatic variables) and active method call frames.
- **Lifecycle:** Memory is allocated the moment a method is invoked and automatically reclaimed as soon as that method finishes executing and returns.
- **Characteristics:** Extremely fast access due to its LIFO (Last In, First Out) structure, but limited in total size. Stack overflow occurs if this limit is exceeded.

2. The Heap

- **What lives here:** All objects and their associated instance variables are stored on the heap.
- **Lifecycle:** An object remains on the heap as long as at least one active reference variable points to it. It has no fixed deallocation point.
- **Garbage Collection:** When an object no longer has any references pointing to it, it becomes eligible for garbage collection. The JVM's Garbage Collector will eventually reclaim that memory to free up heap space. The programmer does not control exactly when this happens.

Part 3: Data Types and Variable Scopes

1. The 8 Primitive Types (Ranked by Memory Size)

Memorize the sizes and ranges below. Exam questions frequently ask which type is most resource-efficient for a given scenario.

- **byte:** 1 byte (8 bits). Range: -128 to 127. Best used for ages, small counts, or raw binary data.
- **short:** 2 bytes (16 bits). Rarely used in modern code but still testable.
- **char:** 2 bytes (16 bits). Stores a single Unicode character (e.g., 'A', '#').
- **int:** 4 bytes (32 bits). The standard default type for whole numbers in Java.
- **float:** 4 bytes (32 bits). Used for single-precision decimals. Requires an f suffix when declared as a literal (e.g., 3.14f).
- **long:** 8 bytes (64 bits). Used for very large integers. Requires an L suffix (e.g., 10000000000L).
- **double:** 8 bytes (64 bits). The default for decimal numbers. Offers higher precision than float.
- **boolean:** Size is not strictly defined by the JVM specification. Stores only the values true or false.

2. Type Casting Mechanics

- **Widening (Implicit):** Moving a value from a smaller type to a larger type (e.g., int to long or float to double). Java performs this automatically because no data can be lost in the conversion.
- **Narrowing (Explicit):** Moving from a larger type to a smaller type (e.g., double to int). You must manually insert a cast operator such as (int). Critically, this truncates the decimal portion rather than rounding. For example, (int)3.99 produces 3, not 4.

3. Variable Scopes

- **Local (Automatic):** Declared inside a method. Java assigns no default value; you must explicitly initialize a local variable before reading it or the compiler will produce an error.
- **Instance Variables:** Declared inside a class but outside any method. Each object created from the class gets its own independent copy. Java auto-initializes these to 0 for numeric types, false for booleans, and null for object references.
- **Class Variables (Static):** Declared with the static keyword. Only one copy exists for the entire class and is shared by all instances. Changing it in one place changes it everywhere.

Part 4: The Four Pillars of OOP

1. Encapsulation (Data Hiding)

Encapsulation is the practice of bundling data (fields) and the code that operates on that data (methods) into a single unit, then restricting direct access to the internals.

- **Implementation:** Declare instance variables as private and provide public getter and setter methods for controlled access.
- **Strictest Control:** The private access specifier is the most restrictive. It limits access exclusively to within the class that declared the member.
- **Validation:** Setters allow you to embed validation logic (for example, rejecting a negative age value) before a change is actually applied, protecting data integrity.

2. Inheritance (The IS-A Relationship)

Inheritance allows one class (the subclass or child) to acquire the properties and behaviors of another class (the superclass or parent), promoting code reuse.

- **Keywords:** Use `extends` for class-to-class inheritance. Use `implements` to inherit from an interface.
- **Single vs. Multiple:** Java supports single inheritance for classes: a class can have only one direct parent. However, a class can implement multiple interfaces, which provides multiple inheritance of type.
- **Non-Inherited Members:** A superclass's private members and its constructors are not inherited by the subclass.
- **The Root Class:** Every class in Java implicitly inherits from `java.lang.Object` if no other parent class is explicitly specified. This is why methods like `toString()` and `equals()` are available on all objects.

3. Polymorphism (Many Forms)

Polymorphism allows a single interface to represent different underlying implementations, enabling more flexible and extensible code.

- **Method Overloading (Compile-time Polymorphism):** Defining multiple methods within the same class that share the same name but have different parameter lists. The correct method is resolved at compile time. This is also known as static binding.
- **Method Overriding (Run-time Polymorphism):** A subclass provides its own specific implementation of a method already defined in its superclass, using the exact same method signature. The correct version is resolved at runtime based on the actual object type. This is known as dynamic binding.
- **Upcasting:** A superclass reference variable can legally point to a subclass object (e.g., `Animal a = new Dog();`). The reverse, assigning a superclass object to a subclass reference, requires an explicit cast and can fail at runtime if the types are incompatible.

4. Abstraction

Abstraction hides the complex internal implementation of a system and exposes only the essential functionality the user needs to interact with.

- **Abstract Classes:** A class declared with the abstract keyword cannot be instantiated directly. It serves as a blueprint, defining a contract that all subclasses must fulfill by implementing the abstract methods.

Part 5: Constructors and the Object Lifecycle

1. Constructors

A constructor is a special block of code that runs automatically when a new object is created, and is used to set its initial state.

- **Rules:** A constructor must have exactly the same name as its class and declares no return type, not even void.
- **Default Constructor:** If you do not define any constructor in a class, the Java compiler silently inserts a no-argument default constructor for you. The moment you write any constructor yourself, the compiler no longer provides this default.
- **Constructor Overloading:** You can define multiple constructors in a class as long as each has a distinct parameter list. This allows objects to be created with different initial configurations.

2. The `this` and `super` Keywords

- **this:** Refers to the current object instance. It is most commonly used to resolve naming conflicts (shadowing) where a method parameter has the same name as an instance variable, for example: `this.name = name;`
- **super:** Refers to the immediate parent class. It can be used to call a parent class method that has been overridden, or to explicitly invoke a parent constructor using `super()`.
- **Placement Rule:** If either `super()` or `this()` is used in a constructor to delegate to another constructor, it must appear as the very first statement in that constructor body. Only one of them can be used per constructor.

Part 6: Access Specifier Visibility

This table summarizes exactly where each access specifier permits visibility. Memorize the progression from most restrictive (private) to least restrictive (public).

Access Specifier	Same Class	Same Package	Subclass (Diff. Package)	Entire World
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Key insight: Each level adds exactly one additional zone of visibility. private is confined to the class. default adds the package. protected adds subclasses in other packages. public removes all restrictions entirely.